

DESIGN AND IMPLEMENTATION BRIEF

Tensor Shape Contracts in GiGL

Jaxtyping contracts, enforced automatically during tests

The problem

Python's `Tensor` annotation says nothing about dtype, rank, or how dimensions relate. In graph learning, a mismatched embedding width or malformed ID tensor may travel through several layers before failing. The eventual error is often far from the boundary that admitted the bad value.

Jaxtyping turns those assumptions into readable, runtime-checkable contracts:

```

from jaxtyping import Float
from torch import Tensor

def decode(
    query: Float[Tensor, "queries embedding_dim"],
    candidate: Float[Tensor, "candidates embedding_dim"],
) -> Float[Tensor, "queries candidates"]: ...

```

The shared `embedding_dim` name requires both inputs to have the same final dimension. A test fails at `decode`, before an invalid tensor can propagate.

Core idea A shape contract records the stable interface: tensor kind, dtype family, rank, and meaningful relationships between axes.

Contract vocabulary

Axes are whitespace-separated. Reusing a name binds equal sizes within one checked call.

Contract	Meaning
<code>Float[Tensor, "batch 128"]</code>	Rank-2 tensor with a fixed final width of 128.
<code>Float[Tensor, "... channels"]</code>	Any number of leading axes, followed by <code>channels</code> .
<code>Float[Tensor, "#batch features"]</code>	<code>batch</code> may match its bound size or be 1 for broadcasting.
<code>Float[Tensor, "{width}"]</code>	Exact size taken from the runtime argument <code>width</code> .
<code>Float[Tensor, "*batch channels"]</code>	Named, variable number of leading axes.

What contracts provide

Benefits

- Express the boundary more precisely than `Tensor`.
- Validate dtype, rank, fixed axes, and named-axis equality.
- Make semantic coupling visible in code review.
- Fail close to the invalid call on exercised test paths.

Limits

- Validate only annotated boundaries reached by tests.
- Cannot model key-to-axis equality in dynamic keyed containers.
- Require accurate contracts; guessed semantics create false failures.
- Complement static typing rather than replace it.

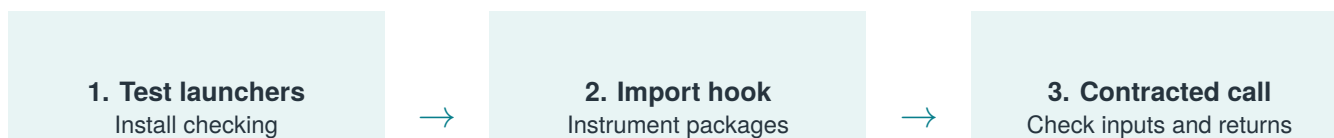
GiGL INTEGRATION

Automatic enforcement in tests

No per-function setup and no production instrumentation

How PR #748 works

Runtime checking is installed by GiGL's unit, integration, and end-to-end test launchers before test discovery. Annotated `gigl` and `examples` code is instrumented as it is imported.



An uncaught `jaxtyping.TypeCheckError` fails the test command. Production training and inference retain the annotations as documentation but do not install runtime checking, avoiding instrumentation overhead and import-order sensitivity.

Why Typeguard 2.x Jaxtyping documents this integration path. GiGL's adapter presents Typeguard with shape-bearing annotations only, avoiding runtime evaluation of static-only Protocol, TypeVar, and forward-reference annotations.

Contracted boundaries

Boundary	Examples
Loaders and samplers	Node IDs, ABLP seeds and labels, loader inputs, and PPR degrees.
Public models and decoders	forward and decode embeddings, shared embedding widths, and score matrices.
Losses and task results	Scores, IDs, and embedding containers with meaningful empty-path ranks.
Examples	Public link-prediction and node-classification model methods.

Contract policy

- Annotate stable tensor boundaries: loader and sampler interfaces, public model `forward` and `decode` methods, loss interfaces, and result containers.
- Use exact dtypes only when the boundary guarantees them. Use `Float` when mixed precision is valid; use `Int64`, `Int32`, `Float32`, or `UInt8` for hard guarantees.
- Reuse axis names only when dimensions must match. Use `_name` for an independent dimension, `_` for an unknown dimension, and `#name` when size 1 is valid for broadcasting.
- Leave internal tensor operations, low-level message passing, and dynamic PyG or TorchRec keyed containers uncontracted unless they expose a stable boundary.
- Prefer no annotation over a guessed rank, dtype, or axis meaning.